Retention Before Retrieval: Why Graph-Aware Memory Eviction Matters for LLM Agents

Minkyu Song
Department of Computer Science, Yonsei University
smkgenesis@yonsei.ac.kr

Abstract

Budget-constrained LLM systems face two logically distinct memory problems: what to retrieve and what to retain. The retrieval problem has received extensive attention; the retention problem has not. We argue that retention is logically prior, because once a structurally necessary block has been evicted, no retrieval method can recover it from the active prompt-visible store.

We study that retention stage with *Dependency-aware Semantic Garbage Collection* (DSGC), a minimal policy that augments query relevance with one-hop prerequisite propagation over a benchmark-supplied dependency graph. The goal is not to present a general agent-memory architecture, but to establish a clean base case for graph-aware retention. The experimental package is fully fixed and public: after an initial preregistered run and documented postmortem, we redesign the benchmark, calibrate template families, freeze a paper-facing main suite, and then run a final robustness pack. In the main suite, all methods solve the control templates perfectly, while DSGC shows large gains on structurally indirect targets, where the needed prerequisite block is not itself strongly similar to the query. Under the lexical feature-hashing encoder, target full-chain retention is 0.90 for DSGC versus 0.03 for retrieval-only and no-graph baselines. Under the sentence encoder, the gap is 1.00 versus 0.23. Robustness experiments show that the gain persists across a moderate λ_π range, appears most clearly at intermediate budgets, and weakens at larger scale for lexical targets. The resulting claim is that one-hop dependency propagation is sufficient to change retention behavior in a benchmarked indirect-prerequisite regime. This paper therefore proposes a framing as well as a result: retention before retrieval should be treated as a first-class memory problem.

1 Introduction

Long-horizon LLM systems routinely compress context because prompt space is limited. Common baselines retain recent tokens, retrieve past text by similarity, or page memories in and out heuristically. These strategies often work well when the needed evidence is either recent or lexically similar to the current query. They are less reliable when success depends on an *older prerequisite* that remains structurally necessary while becoming semantically indirect with respect to the final query.

We refer to such a block as a *structurally indirect prerequisite*: a block whose text is not itself strongly aligned with the query, but whose retention is necessary for recovering the correct answer.

This paper studies that specific failure mode. The central premise is that memory quality is not only a retrieval problem; it is also a retention problem. Most existing approaches effectively frame memory as a retrieval problem: given a query, search a store that is assumed to still contain the needed evidence. Budget-constrained LLM systems face an earlier failure point. Evidence can be permanently discarded before the query arrives, and once an upstream prerequisite has been evicted, no retrieval method can recover it from the active prompt-visible store. Retention is therefore logically prior to retrieval in the failure chain: if retention fails, retrieval operates on an incomplete memory state. We study that retention stage directly rather than proposing another retrieval variant.

We therefore study a minimal dependency-aware policy, DSGC, that adds a one-hop propagation term to similarity-based retention. The policy uses no learned controller, no multi-hop diffusion, and no hidden graph

recall. The question is whether the smallest nontrivial graph-aware retention rule already changes behavior. In that sense, the paper is best understood as a base-case study for graph-based memory management under a broader framing: *retention before retrieval*.

The paper makes four concrete contributions.

1. It formulates an empirical claim around *structurally indirect prerequisites*, positioning DSGC as a base case for graph-aware retention.
2. It designs a benchmark and two-stage calibration-then-freeze protocol that cleanly separates retrieval-friendly controls from dependency-sensitive targets.
3. It shows that in the frozen main suite, retrieval-only and no-graph DSGC coincide, while one-hop DSGC yields large target gains with no control advantage.
4. It documents robustness and limitations explicitly, including budget curves, λ_π sensitivity, a sliding-window baseline, and mixed scale behavior at larger block counts.

2 Retrieval vs. Retention

2.1 The Retrieval Paradigm

Our setting is adjacent to retrieval-augmented language modeling and question answering, including RAG, REALM, RETRO, and Atlas [8, 4, 2, 5]. These systems focus on augmenting generation with external retrieval, typically to improve factual access over large corpora. They answer the question "how should a model fetch supporting evidence?" Crucially, they assume that the relevant evidence is still present in the store at query time. Our question is earlier in the pipeline: under a fixed prompt budget, what should remain in memory before retrieval is even attempted? If a structurally necessary block has already been evicted, better retrieval cannot recover it from the active store.

The paper is also related to graph-augmented retrieval and knowledge access, including GraphRAG and HippoRAG [3, 7]. Those methods use graph structure to expand, organize, or rank retrieved evidence over a store that is again assumed to be complete. DSGC studies a qualitatively earlier use of graph structure: not how to traverse a store more effectively, but how to decide which nodes to keep before budget pressure makes later retrieval impossible. It therefore asks whether the minimal graph-aware retention rule, namely one-hop prerequisite propagation, already changes prompt-visible memory behavior.

2.2 The Retention Gap

Agent-memory systems and long-horizon agent architectures provide a second point of contact. MemGPT, SCM, Generative Agents, Reflexion, Voyager, and LATS all show that external memory, reflection, and planning are central to extended agent behavior [11, 9, 12, 13, 14, 15]. However, these systems do not isolate the specific mechanism studied here: preserving an upstream prerequisite whose text is weakly aligned with the final query. They demonstrate that memory management matters, but they do not separate the retention decision from the broader agent loop. Our contribution is therefore not a broader agent architecture, but a controlled base-case study of the retention decision itself, isolated from the rest of the agent loop.

Finally, the work touches context compression and prompt reduction methods such as Gisting and LongLLMLingua [10, 6]. Those methods ask how to fit more information into a bounded context by compressing or summarizing prompts. They change the representation of what is kept, but they do not directly study which blocks should survive an eviction decision under structural dependence. At a conceptual level, the work is also related to curriculum-style prerequisite structure and staged task decomposition [1]. DSGC addresses that dependence not through training order, but through prompt-level eviction decisions at inference time.

2.3 This Work: Retention as a First-Class Problem

The central claim of this paper is not that retrieval is unimportant, but that it is incomplete as a framing for memory management under budget pressure. Retention and retrieval are distinct stages with different failure modes. Existing work has concentrated overwhelmingly on retrieval over stores that still contain the needed evidence. This paper argues that retention deserves independent study because store incompleteness can arise *before* retrieval is ever invoked. DSGC is therefore presented as a first instance within a broader retention-before-retrieval framework rather than as a complete memory architecture.

3 Method

Problem statement. Given a memory store of N blocks, a query, and a token budget $B < \sum_i s_i$, select a subset of blocks whose total cost does not exceed B. The goal is to maximize the probability that all structurally necessary blocks are retained in the prompt-visible store. This is a retention problem, not a retrieval problem: once a block has been discarded under the budget, it cannot be retrieved later from the active memory state.

3.1 Blocks and Graph

At a query step, the memory store contains blocks $\mathcal{B} = \{b_1, \ldots, b_N\}$. Each block has text, a token cost, and an optional set of prerequisite edges supplied by the benchmark. An edge (j, i) means that block j depends on block i, so i is a prerequisite of j.

The benchmark provides the graph. This isolates the retention question from graph induction. We do not claim that real agent traces already come with clean prerequisite edges; we ask what graph-aware memory management can already achieve once such structure is available.

3.2 Encoders

We evaluate two encoders already implemented in the public codebase. The first is a deterministic 256-dimensional feature-hashing encoder over alphanumeric tokens. It is simple and stable for local reproduction, and it serves as a lexical baseline whose behavior is easy to audit. The second is a sentence encoder backed by `sentence-transformers` using `all-MiniLM-L6-v2`, producing 384-dimensional normalized embeddings. The paper uses both because some failures are mostly lexical, while others remain structural even under a stronger semantic encoder.

3.3 Scoring Rule

For query embedding $\mathbf{g}$ and block embedding $\mathbf{e}_i$, we compute raw relevance scores with a dot product and normalize them with a softmax:

$$r_i = \mathrm{softmax}\left(\frac{\mathbf{g}^\top \mathbf{e}_i}{\tau}\right).$$

In all paper-facing experiments, the relevance temperature is fixed at $\tau = 0.25$.

The only graph term used in the paper-facing policy is one-hop prerequisite propagation:

$$\pi_i = \sum_{j:(j,i)\in E} r_j.$$

The DSGC score is then

$$I_i = r_i + \lambda_\pi \pi_i.$$

Intuitively, if a downstream block j is highly relevant to the current query, its prerequisite i inherits part of that relevance through π_i. This is precisely the intended rescue mechanism: a structurally necessary block should not be discarded merely because its own text is only weakly aligned with the query.

In the main suite, $\lambda_\pi = 1.0$. The no-graph ablation uses the same implementation with graph propagation disabled, so the comparison is *within the same code path*. This was an explicit design requirement after the earlier repository iterations, because a base-case claim is only credible if the graph term is isolated cleanly.

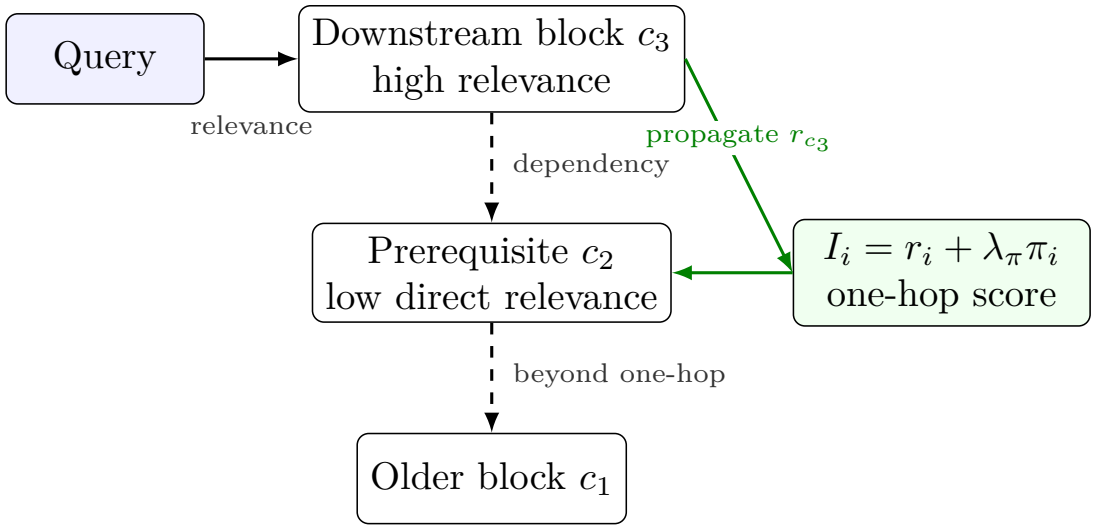

Figure 1: DSGC mechanism. A query-relevant downstream block can rescue its immediate prerequisite through one-hop propagation; the policy does not extend to multi-hop propagation.

3.4 Budgeted Selection

In all experiments, the density exponent is fixed at $\alpha = 0$, so blocks are selected greedily in descending order of I_i until the L1 budget is exhausted. The only policy difference between DSGC and the no-graph ablation is the one-hop propagation term.

This base-case formulation also keeps the cost structure simple. If there are N candidate blocks with embedding dimension d, relevance scoring is a matrix-vector style operation with cost $O(Nd)$. One-hop propagation adds a sparse graph pass with cost $O(E)$ over dependency edges, and final selection adds $O(N \log N)$ under full sorting or a standard top-k routine of comparable practical cost. The complete score-to-selection cycle is therefore transparent: $O(Nd + E + N \log N)$. The paper thus studies the smallest graph-aware intervention that is both empirically identifiable and computationally lightweight.

4 Benchmark and Protocol

4.1 Template Families

The benchmark contains control and target families.

Controls. `budget_control` and `numeric_threshold_control_v2` are retrieval-friendly. They are designed so that the final query directly overlaps with the chain blocks it needs, making graph propagation unnecessary in the clean case. In `budget_control`, one block gives a project budget and another gives the approval margin relative to that budget; the final query simply asks whether a proposed cost is allowed. Because the project name, budget quantity, and approval language all reappear in the query, similarity-only retrieval is already enough. `numeric_threshold_control_v2` plays the same role with threshold-style wording: the decisive numeric rule is itself query-facing, so the benchmark behaves like a standard retrieval problem rather than a dependency-sensitive one. In the redesign phase, a third candidate control (`permission_control_v2`) was rejected because retrieval-only solved it too inconsistently to qualify as a clean control. We keep that rejection in the repository history rather than silently removing it.

Targets. `opaque_role_target_v2` and `opaque_profile_target_v2` are structurally indirect. They require keeping an upstream block whose role is necessary for the answer but not directly favored by similarity alone. In `opaque_role_target_v2`, the query asks whether a user may delete the backup, so the downstream decision block is query-relevant, the middle mapping block is rescued by one hop, and the oldest assignment block still survives by shared keywords. This makes it the cleanest target: the graph term matters, but the oldest block still gets some lexical help. In `opaque_profile_target_v2`, by contrast, the query is about

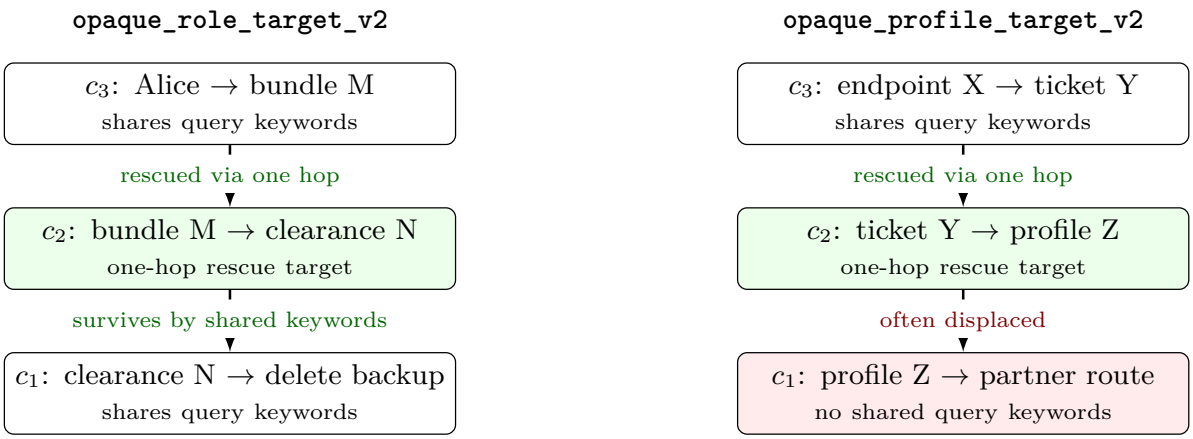

Figure 2: Why the two target templates differ. In `opaque_role_target_v2`, the query about whether a user may delete the backup keeps the downstream decision block relevant, rescues the middle mapping block through one-hop propagation, and still gives the oldest assignment block enough lexical support to survive. In `opaque_profile_target_v2`, the query about direct partner routing again rescues the middle block, but the oldest service-to-ticket block receives little direct support and is therefore the first block to drop.

direct partner routing while the oldest block only links the service to an intermediate ticket, so that earliest prerequisite receives much weaker direct support and is the first block likely to be displaced under budget pressure. This is why the profile template is the hardest target in the suite and the best place to observe residual one-hop failure. Figure 2 illustrates this contrast directly. The final v2 templates were renamed and rebuilt after a postmortem on earlier encoder-sensitive artifacts.

Each scenario also contains *honeypots*: distractor blocks designed to share some query vocabulary without completing the correct reasoning chain. They are necessary because, without such distractors, the benchmark would collapse into trivial recency or lexical retrieval. The calibration stage varied honeypot count and strength, and the frozen main suite uses the medium-strength setting selected from that calibration.

The resulting four-template suite is intentionally small. The controls establish that the benchmark can behave like ordinary similarity-based retrieval when the decisive rule is query-facing. The targets then isolate the opposite regime: the answer still depends on a short chain, but one prerequisite becomes structurally necessary before it becomes lexically attractive. This control-target symmetry matters more for the paper than raw template count. The point is not to enumerate many superficially different tasks, but to pin down a narrow causal contrast in which one-hop graph bias should matter only in the indirect cases and should disappear in the direct ones.

4.2 Experimental History

The public codebase preserves the experiment history instead of exposing only the winning run.

1. `final_v1`: initial preregistered four-experiment suite.
2. `final_v1` postmortem: documented failure analysis and redesign rationale.
3. `final_v2` calibration: used to select clean controls, target families, and operating regimes.
4. `final_v2_main`: frozen paper-facing main suite.
5. `final_v3_robustness`: final sensitivity and limitation pack.

This sequence matters because the paper is not claiming that the first benchmark draft was already correct. The redesign history is part of the scientific record, not an inconvenience to be hidden.

4.3 Frozen Main Suite

The fixed `final_v2_main` protocol uses:

- 4 templates: 2 controls and 2 targets;
- methods: retrieval-only, no-graph DSGC, and DSGC;
- encoders: frozen and sentence;
- 15 seeds;
- total block count 20;
- medium-strength honeypots;
- control settings: chain depth 2, 4 honeypots, budget multiplier 2.0;
- target settings: chain depth 3, 2 honeypots, budget multiplier 2.0.

The primary metric is `full_chain_retention`. We also track prerequisite retention and answer accuracy, but in the final suite answer accuracy is effectively redundant with full-chain retention and is treated as secondary.

The preregistered predictions were:

1. controls: retrieval-only should achieve at least 0.90 and DSGC should match it;
2. targets: DSGC should outperform both retrieval-only and no-graph DSGC;
3. `opaque_role_target_v2` should remain the cleanest target;
4. `opaque_profile_target_v2` should preserve a positive DSGC advantage under the sentence encoder.

5 Main Results

Table 1 shows the paper-facing regime averages. The control story is completely clean: all three methods achieve perfect full-chain retention under both encoders. The target story is equally clear: retrieval-only and no-graph DSGC coincide, while DSGC is much stronger.

Encoder	Regime	Method	Full-chain retention
Frozen	Control	Retrieval-only	1.0000 ± 0.0000
Frozen	Control	No-graph DSGC	1.0000 ± 0.0000
Frozen	Control	DSGC	1.0000 ± 0.0000
Frozen	Target	Retrieval-only	0.0333 ± 0.1826
Frozen	Target	No-graph DSGC	0.0333 ± 0.1826
Frozen	Target	DSGC	0.9000 ± 0.3051
Sentence	Control	Retrieval-only	1.0000 ± 0.0000
Sentence	Control	No-graph DSGC	1.0000 ± 0.0000
Sentence	Control	DSGC	1.0000 ± 0.0000
Sentence	Target	Retrieval-only	0.2333 ± 0.4302
Sentence	Target	No-graph DSGC	0.2333 ± 0.4302
Sentence	Target	DSGC	1.0000 ± 0.0000

Table 1: Regime-level full-chain retention in the fixed `final_v2_main` suite.

The template-level picture is consistent with the preregistration and also shows that the profile target is harder than the role target.

Encoder	Target template	Retrieval-only	No-graph DSGC	DSGC
Frozen	`opaque_role_target_v2`	0.0000 ± 0.0000	0.0000 ± 0.0000	1.0000 ± 0.0000
Frozen	`opaque_profile_target_v2`	0.0667 ± 0.2582	0.0667 ± 0.2582	0.8000 ± 0.4140
Sentence	`opaque_role_target_v2`	0.1333 ± 0.3519	0.1333 ± 0.3519	1.0000 ± 0.0000
Sentence	`opaque_profile_target_v2`	0.3333 ± 0.4880	0.3333 ± 0.4880	1.0000 ± 0.0000

Table 2: Target-template full-chain retention in the main suite (mean ± standard deviation over 15 seeds).

Two aspects of Table 1 matter most.

First, the control result rules out the trivial interpretation that DSGC simply adds a blanket advantage everywhere. It does not. In retrieval-friendly settings, DSGC has no measurable edge because there is no structural dependency for the graph term to rescue.

Second, the near-exact equality between retrieval-only and no-graph DSGC is important. This equality is not accidental. With graph propagation disabled, the policy score reduces to $I_i = r_i$, and r_i is just a softmax over raw dot-product scores. Since softmax preserves ranking, no-graph DSGC and retrieval-only induce the same block ordering. The gain is therefore not coming from an unrelated implementation difference; it is tied to the one-hop propagation term itself, which is exactly what a base-case graph-memory argument requires.

6 Robustness and Limitations

6.1 Sensitivity and Budget Curves

The `final_v3_robustness` pack adds four checks: a λ_π sweep, a budget curve, a sliding-window baseline, and a scale test.

Across $\lambda_\pi \in \{0.5, 1.0, 2.0\}$, target gains remain positive. On frozen `opaque_profile_target_v2`, DSGC moves from 0.6667 to 0.8000 to 0.8667 as λ_π increases, while retrieval-only remains at 0.0667. On frozen `opaque_role_target_v2`, DSGC remains at 1.0000 across all three values while retrieval-only stays at 0.0000.

The budget curve reveals the intended operating-range story. At tight budgets, all methods may fail because there is simply not enough room for the chain. At intermediate budgets, DSGC separates sharply from retrieval-only. At loose budgets, retrieval-only and no-graph DSGC begin to catch up. For instance, on frozen `opaque_profile_target_v2`, DSGC improves from 0.5333 at multiplier 1.0 to 0.8000 at 2.0 and 1.0000 at 5.0, whereas retrieval-only moves from 0.0000 to 0.0667 to 0.6000 over the same budgets. The retrieval-only curve is not perfectly monotone: at multiplier 5.0 on frozen `opaque_role_target_v2`, the larger budget admits additional filler and honeypot blocks whose frozen-encoder scores remain competitive with the chain, so a chain block can still be displaced even after a smaller budget happened to retain it. This is the narrow operating-range claim of the paper: dependency-aware retention matters most before the budget becomes loose enough to keep nearly everything anyway.

6.2 Sliding Window Baseline

The sliding-window baseline remains weak throughout the budget sweep. At budget multiplier 2.0, it reaches only 0.0667 on all four frozen templates, including the controls. This is not the primary scientific comparison, but it answers the obvious reviewer question about a naive recency baseline. The failure is expected: in these tasks, recency is neither equivalent to relevance nor sufficient to preserve the prerequisite chain. We do not include a random selector because it would be strictly weaker than both recency and similarity baselines and therefore adds little diagnostic value beyond those stronger comparators.

6.3 Scale

The strongest limitation is scale. Controls remain perfect at 50 total blocks, and sentence-encoder DSGC stays strong on the targets. The frozen-encoder targets do not. In the frozen setting, the profile target drops from 0.8000 at 20 blocks to 0.3333 at 50 blocks, and the role target drops from 1.0000 to 0.7333. A plausible mechanism is that, as the candidate pool grows, relevance mass spreads across more distractors and the one-hop propagation signal has to compete with a larger set of semantically similar blocks. We therefore do *not* claim that the current method is scale-robust in general.

6.4 Failure Diagnosis

The codebase records rank and displacement diagnostics for each run. In the three frozen profile-target failures of the main suite, the immediate issue is displacement of the oldest prerequisite block `c1`. In those failed seeds, `c2` and `c3` rank first and second, while `c1` falls to rank 10, 6, and 6 respectively and is displaced by non-chain blocks. This matters because it grounds the remaining failure in public diagnostics rather than in post hoc speculation.

7 Discussion

The paper supports the following empirical statement: Under fixed prompt budgets, one-hop dependency propagation is sufficient to improve retention when the answer depends on structurally necessary but semantically indirect prerequisites.

This positions the contribution at the retention stage of memory management, which is logically prior to retrieval. Much of the existing literature has focused on retrieval: how to find relevant evidence in a store that still contains it. The present results suggest that retention deserves independent study, because structural prerequisites can be lost before retrieval is ever invoked.

This is not a claim that "graph memory beats retrieval" or that "DSGC solves long-horizon memory." What the paper establishes is a constructive first case: once structural dependence is made explicit, even one-hop graph bias changes retention behavior in a way that similarity-only policies do not match. Several scope boundaries apply.

- The benchmark uses supplied dependency graphs rather than inferred ones.
- The strongest evidence comes from synthetic template families chosen after an explicit redesign process.
- Robustness at larger block counts is mixed, especially for the frozen encoder.
- The answer metric is not independent of full-chain retention in the final suite.

At the same time, the evidence is stronger than a single benchmark win. The repository includes a failed first version, a postmortem, a calibration run, a frozen main suite, and a separate robustness pack. That visible iteration history is part of why the claim is credible.

The largest practical limitation is that the experiments assume supplied dependency graphs. Future work could therefore explore lightweight graph induction over agent traces, multi-hop propagation, and incremental graph-aware memory stores. More broadly, the retention-before-retrieval framing suggests inferring dependency edges from traces, testing whether multi-hop propagation helps at larger scale, co-optimizing retention and retrieval, and adapting policies to changing budgets. The broader implication is not that graph memory is solved, but that even minimal structural awareness is already strong enough to justify pursuing those richer directions.

7.1 Hardware and Systems Considerations

The one-hop restriction is not only a conceptual choice; it also preserves a favorable systems profile. The relevance term r_i is a dense query–block inner product, the propagation term π_i is a single sparse pass implementable as SpMV or scatter-add, and final retention reduces to score ordering plus a standard top-k budgeted selection step. The resulting cost is $O(Nd + E + N \log N)$, where N is the number of candidate blocks, d is embedding dimension, and E is the number of dependency edges. When the graph is sparse, the extra structural cost follows declared prerequisite edges rather than an $O(N^2)$ all-pairs interaction pattern. That does not make DSGC universally cheaper than every alternative, but it does make the base case algorithmically lightweight enough to be plausible as an online memory-management component.

Moving beyond one hop would preserve those sparse primitives but lose the single-pass property. A k-hop DSGC would look more like iterative SpMV or repeated scatter-add, pushing propagation toward $O(kE)$, increasing synchronization overheads, and introducing possible score amplification or diffusion artifacts that may require damping or normalization. We therefore treat multi-hop DSGC not as a free extension, but as a separate systems problem with new efficiency and stability tradeoffs.

We do *not* benchmark kernel-level latency, throughput, or memory overhead on a production inference stack. The claim is narrower: the base-case rule is substantially simpler than full graph neural reasoning and structurally more hardware-friendly than repeated global recomputation.

The same separation matters for graph induction. In realistic deployments, edge construction need not sit on the per-step retention hot path; if edges are created when blocks are ingested or updated, the induction cost is amortized over later retention cycles. In trace-rich agents, some dependencies may also be recoverable from execution logs, tool-call traces, or other lightweight monitors rather than from repeated LLM inference. We do not benchmark that path here, but it suggests a plausible deployment route in which graph maintenance remains materially cheaper than repeating a more expensive memory-reasoning procedure at every step.

8 Reproducibility and Open-Source Artifact

The curated public release is published at `https://github.com/smkgenesis/dsgc`; running `reproduce.sh` recreates the paper-facing `final_v2_main` and `final_v3_robustness` outputs and preserves the redesign history.

9 Conclusion

This paper defines a benchmarked indirect-prerequisite failure mode, evaluates a one-hop graph-aware retention rule against matched baselines, and releases the resulting protocol as a public artifact. The main suite cleanly separates retrieval-friendly controls from dependency-sensitive targets, while the robustness pack shows that the effect survives moderate hyperparameter variation and is concentrated in the intermediate-budget regime. The result does not establish general graph-based memory management, but it does establish a transparent empirical base case for graph-aware retention as the logically prior stage in the retention-before-retrieval framing.

References

[1] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48, 2009.

[2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and Laurent Sifre. Improving language models by retrieving from trillions of tokens, 2021.

[3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization, 2024.

[4] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training, 2020.

[5] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models, 2022.

[6] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression, 2023.

[7] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2020.

[9] Xinnian Liang, Bing Wang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li. Unleashing infinite-length input capacity for large-scale language models with self-controlled memory system, 2023.

[10] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens, 2023.

[11] Charles Packer, Vivian Fang, Shishir Patil, Kevin Lin, Sarah Wooders, and Joseph Gonzalez. Memgpt: Towards llms as operating systems, 2023.

[12] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.

[13] Noah Shinn, Federico Cassano, Bailin Labash, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

[14] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.

[15] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models, 2024.

A Target Lambda Sweep

Table 3 gives the target-only λ_π sweep. The role target is saturated throughout, while the frozen profile target improves gradually as propagation weight increases.

Template	Encoder	λ_π	Mean	Std
`opaque_role_target_v2`	Frozen	0.5	1.0000	0.0000
`opaque_role_target_v2`	Frozen	1.0	1.0000	0.0000
`opaque_role_target_v2`	Frozen	2.0	1.0000	0.0000
`opaque_profile_target_v2`	Frozen	0.5	0.6667	0.4880
`opaque_profile_target_v2`	Frozen	1.0	0.8000	0.4140
`opaque_profile_target_v2`	Frozen	2.0	0.8667	0.3519
`opaque_role_target_v2`	Sentence	0.5	1.0000	0.0000
`opaque_role_target_v2`	Sentence	1.0	1.0000	0.0000
`opaque_role_target_v2`	Sentence	2.0	1.0000	0.0000
`opaque_profile_target_v2`	Sentence	0.5	1.0000	0.0000
`opaque_profile_target_v2`	Sentence	1.0	1.0000	0.0000
`opaque_profile_target_v2`	Sentence	2.0	1.0000	0.0000

Table 3: Target-only summary of the λ_π sweep from the robustness pack (mean and standard deviation over 15 seeds).

B Scale Robustness

Table 4 gives the exact target-level numbers for the 20-block and 50-block settings. The sentence encoder remains saturated, whereas the frozen encoder degrades materially on both targets.

Template	Encoder	20 blocks	50 blocks
`opaque_role_target_v2`	Frozen	1.0000 ± 0.0000	0.7333 ± 0.4577
`opaque_profile_target_v2`	Frozen	0.8000 ± 0.4140	0.3333 ± 0.4880
`opaque_role_target_v2`	Sentence	1.0000 ± 0.0000	1.0000 ± 0.0000
`opaque_profile_target_v2`	Sentence	1.0000 ± 0.0000	1.0000 ± 0.0000

Table 4: DSGC full-chain retention under the scale robustness check (mean $\pm$ standard deviation over 15 seeds).

C Failure Diagnosis

Table 5 records the three failing frozen `opaque_profile_target_v2` seeds. In each case, the dropped block is the earliest prerequisite c_1 even though the later chain blocks remain highly ranked.

Seed	Chain ranks $(c1, c2, c3)$	Top non-chain block	Displaced block
3	$(10, 1, 2)$	`h1`	`c1`
8	$(6, 1, 2)$	`f9`	`c1`
14	$(6, 1, 2)$	`f3`	`c1`

Table 5: Frozen `opaque_profile_target_v2` failures from the main suite (3 failures out of 15 seeds).

D Public Artifact Map

The public release at https://github.com/smkgenesis/dsgc is split into headline paper-facing results and historical redesign records. The four headline files cited in the main text live under `results/`: `final_v2_main_summary.json`, `final_v2_main_report.md`, `final_v3_robustness_summary.json`, and `final_v3_robustness_report.md`.

The transparency record referenced in Section 4 is preserved under `audit/results/`, `audit/docs/`, and `audit/protocols/`, including `final_v1_report.md`, `final_v2_calibration_pass1_report.md`, `final_v1_postmortem.md`, `final_v2_design.md`, `final_experiment_v1.json`, and `final_experiment_v2_draft.json`.